Unidad 10 Ficheros Binarios

Introducción
Flujos de Salida Binarios
Flujos de entrada binarios
Ficheros binarios y objetos complejos

Introducción

Ahora nos ocuparemos de los flujos de datos de tipo byte, que nos van a permitir guardar (o transferir) y recuperar (o recibir) cualquier tipo de datos usados en un programa. No olvidemos que, para usar cualquier flujo en Java, debemos importar las clases del paquete java. io.

Cuando se trata de escribir (o leer) bytes en un flujo, existen dos clases básicas, **FileOutputStream** y **FileInputStream**

Pero nosotros no solemos manejar bytes individuales en nuestros programas, sino datos (eso sí, formados por bytes) más complejos, ya sean de tipos primitivos u objetos. Por eso necesitamos un intermediario capaz de convertir los datos complejos en series planas de bytes o reconstruir los datos a partir de series de bytes, en procesos de serialización y de deserialización de datos, respectivamente.

Esos intermediarios son flujos llamados envoltorio: **ObjectOutputStream** y **ObjectInputStream**, que se crean a partir de flujos de bytes planos, como **FileOutputStream** y **FileInputStream**.

Flujos de salida binarios

Supongamos que queremos grabar en disco los enteros guardados en una tabla. Para ello empezaremos creando un flujo de salida de tipo binario, asociado al fichero donde vamos a grabarlos, que llamaremos **enteros.dat**.

```
FileOutputStream archivo = new FileOutputStream("enteros.dat");
```

El constructor puede arrojar una excepción del tipo **FileNotFoundException**, que hereda de **IOException**. La sentencia creará en el disco el archivo enteros.dat. Si ya existía, borrará la versión anterior y lo sustituirá por una nueva.

Una vez creado este flujo, lo «envolvemos» en un objeto de la clase **ObjectOutputStream**.

```
ObjectOutputStream out = new ObjectOutputStream(archivo);
```

El constructor de **ObjectOutputStream** puede arrojar una excepción **IOException** (excepción de entrada-salida). Por tanto, debe ir encerrado en una estructura try-catch, que puede englobar también al constructor del objeto **FileOutputStream**.

La clase **ObjectOutputStream** tiene una serie de métodos que permiten la escritura de datos complejos de cualquier tipo o clase, serializándolos antes de enviarlos al flujo de salida. **Para ello, las clases de estos datos deben tener implementada la interfaz Serializable**, que no es más que una especie de sello que declara a sus objetos como susceptibles de ser serializados, es decir, convertibles en una serie plana de bytes.

Las clases implementadas por Java, como String, las colecciones y las tablas, traen implementadas la interfaz **Serializable**. Los objetos de estas clases, así como los datos de tipo primitivo, son serializados automáticamente por Java. En cambio, las clases definidas por el usuario deben declararse como serializables en su definición, sin que esto nos obligue a implementar ningún método especial.

```
class miClase implements Serializable {
    //cuerpo de la clase
}
```

Con esto, **miClase** ya es serializable, y sus objetos susceptibles de ser enviados por un flujo binario

ObjectOutputStream dispone de los siguientes métodos para la escritura de datos en un flujo de salida:

- void writeBoolean(boolean b): escribe un valor boolean en el flujo.
- void writeChar(int c): escribe el valor char que ocupa los dos bytes menos significativos del valor entero que se le pasa como parámetro.
- void writeInt(int n): escribe un entero.
- void writeLong(long n): escribe un entero largo.
- void writeDouble(double d): escribe un número de tipo double.
- void writeObject(Object o): escribe un objeto serializable.
- …

Ejemplo: Escribir en un archivo **datos.dat** los valores de una tabla de diez enteros.

```
/*Inicializamos la tabla con los enteros del 0 al 9. Luego creamos el archivo y le asociamos
un flujo de salida de la clase ObjectOutputStream. A continuación, recorremos la tabla
escribiendo los enteros en él*/
int[] t = {0, 1, 2, 3, 4, 5, 6, 7, 8, 9};
ObjectOutputStream flujoSalida = null;
try {
    flujoSalida = new ObjectOutputStream(new FileOutputStream("datos.dat"));
    for (int n : t) {
        flujoSalida.writeInt(n);
    }
} catch (IOException ex) {
    System.out.println(ex);
} finally {
    if (flujoSalida != null) {
        try {
                flujoSalida.close();
        } catch (IOException ex) {
    System.out.println(ex);
    }
}
```

Hemos recorrido la tabla para obtener sus elementos y guardarlos por separado. Pero, en Java, una tabla es un objeto, y podríamos haberla escrito en el archivo como tal objeto, usando el método void **writeObject(Object o)**. Por tanto, en este código el bucle for puede ser sustituido por una sentencia

```
flujoSalida.writeObject(t);
```

En este caso grabamos la tabla como objeto, que no es lo mismo que grabar los enteros por separado. Esta distinción será importante a la hora de recuperarla.

Igualmente, para guardar una cadena de caracteres se usa el método **void writeObject(Object o)**, ya que una cadena es un objeto de la clase String.

```
String cadena = "Sancho Panza";
flujoSalida.writeObject(cadena);
```

Ejemplo: Escribe como una cadena, en el fichero binario **cancionPirata.dat**, la siguiente estrofa:

```
Con diez cañones por banda,
viento en popa a toda vela,
no corta el mar, sino vuela
un velero bergantín.
```

```
/*Como no se trata de un archivo de texto, convertimos la estrofa en una cadena,
incluyendo los cambios de línea, y luego la escribimos en el flujo como un objeto
String*/
String estrofa = "Con diez cañones por banda, \n"
     + "viento en popa a toda vela, \n"
     + "no corta el mar, sino vuela \n"
     + "un velero bergantín.";
try ( ObjectOutputStream out = new ObjectOutputStream(new
FileOutputStream("canconPirata.dat"))) {
     out.writeObject(estrofa);
}catch (IOException ex) {
     System.out.println(ex);
}
/*La estrofa se guardará con la codificación específica de los objetos de la clase
String y no podremos leerla directamente del archivo con un editor de texto. Si
queremos guardar texto legible con un editor de texto, deberemos usar archivos de
texto*/
```

Hasta **Java 7**, los flujos, tanto de texto como binarios, había que cerrarlos con el método **close()** ,disponible en todas las clases de entrada y salida, incluyéndolo en un bloque **finally**. No obstante, como vimos con los archivos de texto y en el ejemplo resuelto anterior, **usando una estructura try-catch con recursos, el cierre es automático y NO TENEMOS QUE USAR EL MÉTODO close().**

Flujos de entrada binarios

Para leer de fuentes de datos binarios, usaremos flujos de la clase **ObjectInputStream**, construidos a partir de un flujo de bytes planos **FileInputStream**. Por ejemplo, si leemos los datos escritos en el archivo datos. dat de los ejemplos anteriores, quedaría:

```
ObjectInputStream flujoEntrada = new ObjectInputStream(new
                                 FileInputStream("datos.dat"));
```

Esta sentencia puede producir una excepción **IOException**; por tanto, deberá ir encerrada en una estructura try-catch. Lo mismo ocurre a la hora de cerrarlo con el método **close()**, aunque nosotros usamos habitualmente una estructura **try-catch con recursos**.

Los métodos de la clase **ObjectInputStream** permiten leer los mismos datos que grabamos con **ObjectOutputStream**. Por cada método de escritura de esta última hay otro de lectura de la primera. En el caso de que hayamos grabado los 10 enteros de una tabla por separado usando **writeInt()**, los podemos recuperar, también por separado, con el método **readInt()**,que puede arrojar una excepción **IOException** si hay un error de lectura o **EOFException** si se ha llegado al final del fichero.

Ejemplo: Leer de un archivo **datos.dat** los datos de 10 números enteros, guardándolos en una tabla de tipo int.

```
//usaremos una estructura try-catch con recursos
try (ObjectInputStream flujoEntrada = new ObjectInputStream (
new FileInputStream("datos.dat"))) {
     int[] t = new int[10];
     for (int i = 0; i < t.length; i++) {
          t[i] = flujoEntrada.readInt();
     }
     System.out.println(Arrays.toString(t));
} catch (IOException ex) {
     System.out.println("error lectura");
}
```

Los métodos más importantes de **ObjectInputStream** son los siguientes:

- boolean readBoolean(): lee un booleano del flujo de entrada.
- char readChar(): lee un caracter.
- int readInt(): lee un entero.
- long readLong(): lee un entero largo.
- double readDuoble(): lee un número real de tipo double.
- Object readobject(): lee un objeto.

Dado que las tablas son objetos, si se ha guardado la tabla t usando el método **writeObject()**, en vez de un bucle for para la lectura, usaremos una sentencia única, ya que lo que hay guardado es un objeto, no una serie de enteros.

Ejemplo: Leer una tabla de enteros de un archivo datos.dat

```
try (ObjectInputStream flujoEntrada = new ObjectInputStream(
new FileInputStream("datos.dat"))) {
    int[] tabla = (int[]) flujoEntrada.readObject();
    System.out.println(Arrays.toString(tabla));
} catch (IOException e) {
    System.out.println("Error de entrada/salida");
} catch (ClassNotFoundException e) {
    System.out.println("El fichero no almacena un objeto tabla");
}
```

El cast **(int [ ])** es necesario, ya que **readObject()** devuelve un objeto de la clase **Object**, que es asignado a una variable de tipo **int[ ]** (tabla de enteros), lo que supone una conversión de estrechamiento.

Llama la atención la excepción **ClassNotFoundException**, que puede ser arrojada por el método **readObject()**. Esto se debe a que, cuando leemos un objeto de un flujo de entrada, puede ocurrir que la clase a la que pertenece no sea visible desde el lugar del código donde se invoca **readObject()**, debido a que no esté en el mismo paquete ni haya sido importada de otro.

Como ya vimos, las cadenas de texto son objetos y, si se guardaron como tales, se deben recuperar utilizando el método **readObject().**

```
try {
        String cadena = (String) flujoEntrada.readObject();
} catch (ClassNotFoundException ex) {
        System.out.println(ex.getMessage());
}
```

A menudo desconocemos el número de datos guardados en un archivo. En este caso, para recuperarlos todos, no podemos usar un bucle for controlado por contador, sino que tenemos que leer hasta que se llegue al final del fichero, es decir, hasta que salte la excepción **EOFException**. Por ejemplo, si un fichero contiene una lista de enteros y no sabemos cuántos hay, para recuperarlos todos, usamos un bucle infinito del que solo nos puede sacar la excepción **EOFException** de fin de fichero.

```
try {
    while (true) {
        System.out.println(in.readInt());
    }
}catch(EOFException ex) {
    System.out.println("Fin de fichero");
}
```

Cuando se haya leído el último entero, se habrá llegado al final del fichero. Entonces se arrojará la excepción yel programa saldrá del bucle while ydel bloque try para continuar en el bloque catch.

Ficheros binarios y objetos complejos

Los objetos que queremos guardar en un archivo binario no siempre son tan simples como una cadena de caracteres. La mayoría pertenecen a clases con atributos, que muchas veces son también objetos.

Los valores de estos atributos, en realidad, son solo referencias a los objetos propiamente dichos. Entonces se plantea la siguiente cuestión: ¿qué se guarda en el fichero, el valor del objeto referenciado o solo la referencia? Si fuera solo la referencia, no estaríamos guardando la información que nos interesa, ya que **las referencias cambian en cada ejecución del programa. Si leyéramos el archivo al día siguiente en una nueva ejecución del programa, no recuperaremos la información del objeto, sino la dirección de memoria que tenía cuando se guardó.**

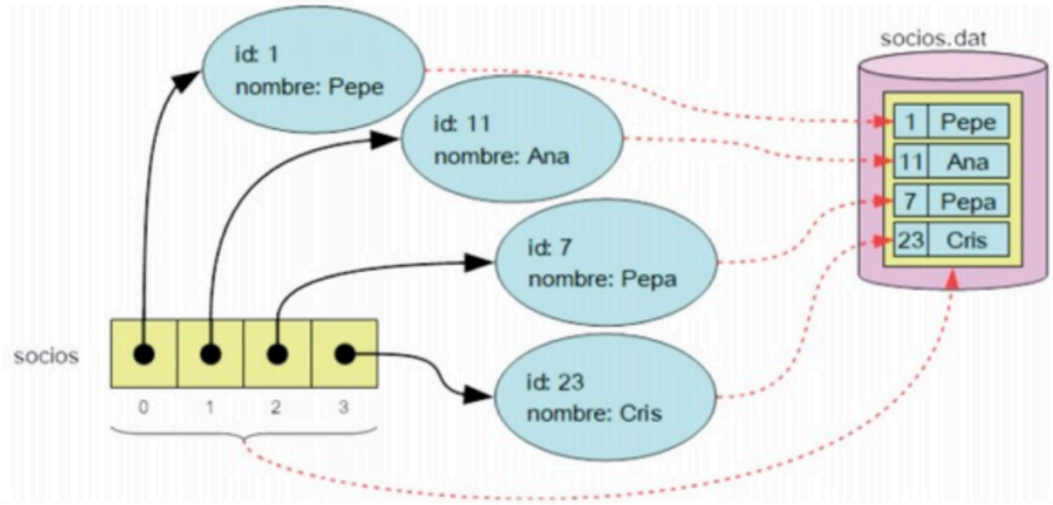
Supongamos que queremos guardar una tabla de objetos de la **clase Socio**. Pasaremos al método **writeObject()** la variable **tablaSocios**, que guarda la referencia a la tabla en la memoria. Pero no olvidemos que cada componente de la tabla guarda, a su vez, la referencia a un objeto de la clase Socio, no el objeto propiamente dicho.

¿Qué se guarda realmente en el archivo? La respuesta es: toda la información necesaria para reconstruir la tabla cuando se vuelva a leer del archivo. Esto incluye: la propia tabla y los objetos referenciados en cada componente, con la información sobre su clase y los valores de los atributos, incluidos aquellos que referencian otros objetos, como el nombre o el dni. Los atributos pueden ser incluso otras tablas, como la lista de los familiares del socio, que se guardarían de la misma forma. Java rastrea todas las referencias a objetos hasta construir la estructura completa de los datos, y guarda toda la información necesaria para reconstruir de nuevo el objeto guardado, junto con todos los objetos referenciados desde él, cuando se lea más tarde con **readObject().**

Ejemplo – Tabla Objetos Socio

Implementar un programa que guarde en el fichero socios.dat una tabla de objetos Socio. Después se abrirá de nuevo el fichero en modo lectura para recuperar la tabla de socios, mostrándose por pantalla.

```
/*Implementaremos una clase Socio simplificada, suficiente para ilustrar lo que
nos interesa. Para que se pueda guardar en un fichero binario, deberá implementar
la interfaz Serializable*/
class Socio implements Serializable {
        String dni;
        String nombre;
        public Socio(String dni, String nombre) {
                this.dni = dni;
                this.nombre = nombre;
        }
        @Override
        public String toString() {
                return "Socio{" + "dni=" + dni + ", nombre=" + nombre + '}';
        }
}
/*El programa principal sería: */
Socio[] tablaSocios = new Socio[4];
tablaSocios[0] = new Socio("1", "pepe");
tablaSocios[1] = new Socio("11", "ana");
tablaSocios[2] = new Socio("7", "pepa");
tablaSocios[3] = new Socio("23", "cris");
//mostramos la tabla de socios antes de guardarla:
System.out.println(Arrays.deepToString(tablaSocios));
//Creamos un flujo de salida binario y escribimos en él:
try(ObjectOutputStream salida=new ObjectOutputStream(new
FileOutputStream("socios.dat"))){
            salida.writeObject(tablaSocios);
}catch (IOException ex) {
            System.out.println(ex);
} //El flujo de salida se cierra automáticamente
/*Creamos un flujo de entrada y leemos de él la tabla de socios, que asignaremos
a la misma variable tablaSocios. El bloque catch recoge tanto la excepción
IOException de la apertura del flujo como ClassNotFoundException ligado al cast
(Socio[]), como ya se comentó en el parágrafo 11.2*/
try(ObjectInputStream entrada=new ObjectInputStream(
new FileInputStream("socios.dat"))){
       tablaSocios=(Socio[])entrada.readObject();
} catch (IOException | ClassNotFoundException ex) {
       System.out.println(ex);
}
//Volvemos a mostrar la tabla de socios:
System.out.println(Arrays.deepToString(tablaSocios));
```



Recuerda que en una estructura try-catch, el bloque catch puede capturar más de un tipo de excepción.

Para ello, basta escribir en el paréntesis todos los tipos separados por barras verticales, poniendo al final el nombre del parámetro que referencia la excepción, que funcionará como variable local dentro del bloque.